INFOSYS SPRINGBOARD 7.0 INTERNSHIP PROGRAM — CAPSTONE SPECIFICATION

PRICEPILOT AI
COMPLETE SYSTEM ARCHITECTURE & DESIGN DIAGRAMS

Machine Learning Based Dynamic Pricing and Demand Forecasting System

Document Title:	PricePilot AI Complete System Architecture & Design Diagrams
Document ID:	DOC-DIAG-PRICEPILOT-2026-V2
Organization:	Infosys Springboard 7.0
Authoring Team:	Narendar Reddy · Manvitha · Pravallika · Ashwindh
Release Version:	Version 2.0.0 Enterprise Production
Completion Date:	August 2026
Verified Source Scope:	FastAPI, React 19 SPA, Extra Trees Regressor, SQLAlchemy, PostgreSQL

2.0 TABLE OF CONTENTS

3.0 PROJECT OVERVIEW & VERIFIED SYSTEM SCOPE

PricePilot AI is an enterprise dynamic pricing optimization and revenue intelligence system engineered to calculate optimal market prices, predict product demand across short/medium/long term horizons, monitor competitor price movements, and enforce profit margin thresholds.

Verified System Implementation:

• **Frontend:** React 19 single-page application built with Vite, utilizing React Router v6, Tailwind CSS, Recharts, and `AuthContext`.
• **Backend:** Python 3.13 FastAPI microservices with SQLAlchemy 2.0 ORM, OAuth2 JWT bearer token security, and openpyxl binary Excel export engine.
• **Machine Learning:** Extra Trees Regressor (`best_price_model.pkl`) benchmarked at $R^2 = 0.9650$ (MAE = 31.1766, RMSE = 108.6525) accepting 16 input attributes.
• **Database:** 11 PostgreSQL / SQLite relational tables (`users`, `products`, `predictions`, `price_recommendations`, `demand_forecasts`, `prediction_history`, `notifications`, `reports`, `activity_logs`, `settings`, `password_reset_otps`).

4.0 SYSTEM ARCHITECTURE DIAGRAMS

4.1 Overall System Architecture Diagram

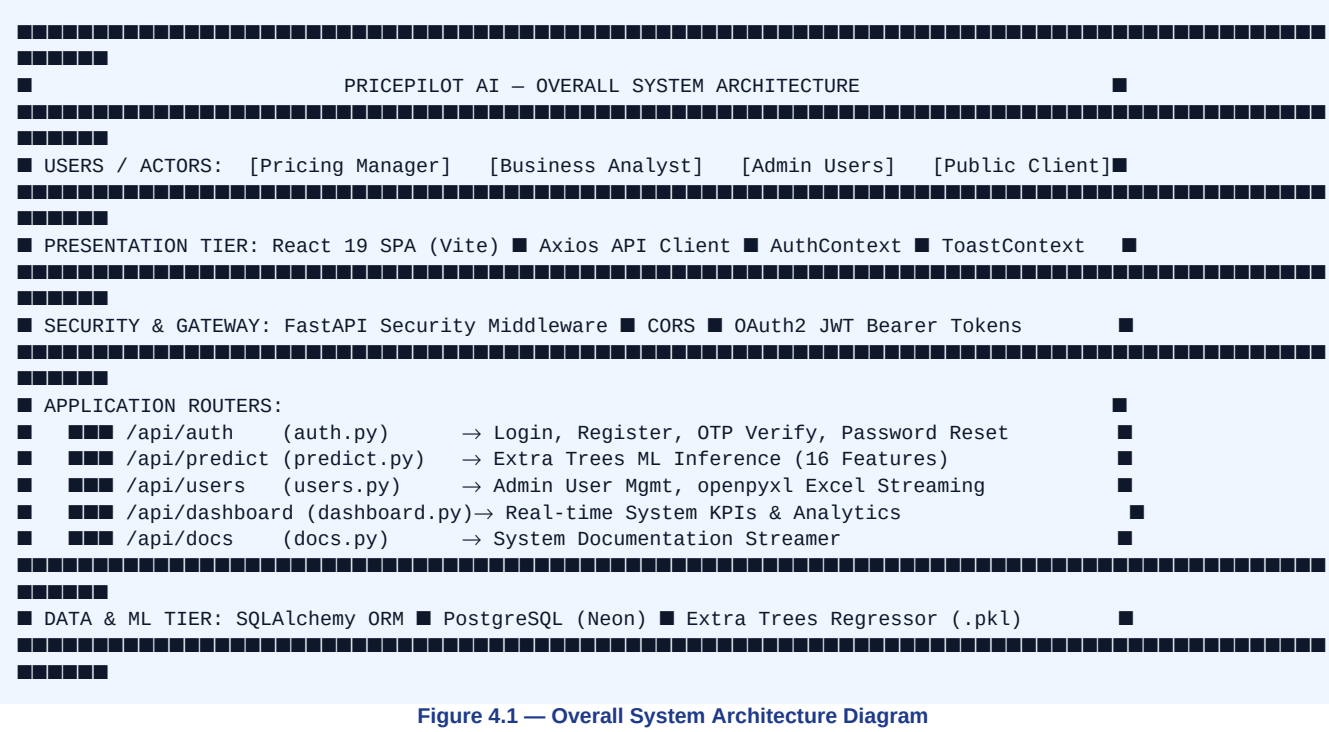

Figure 4.1 — Overall System Architecture Diagram

4.2 Frontend Architecture Diagram

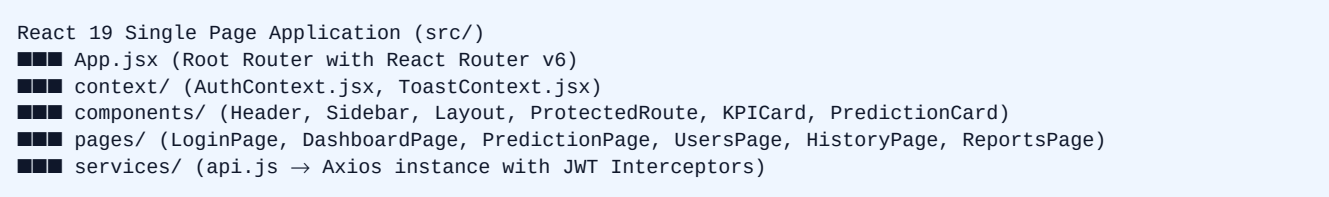

Figure 4.2 — Frontend SPA Architecture Diagram

4.3 Backend Architecture Diagram

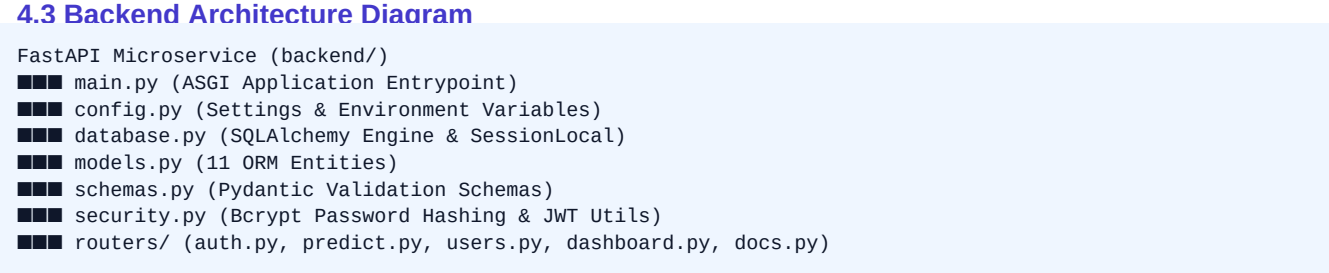

Figure 4.3 — Backend Microservice Architecture Diagram

4.4 Machine Learning Architecture Diagram

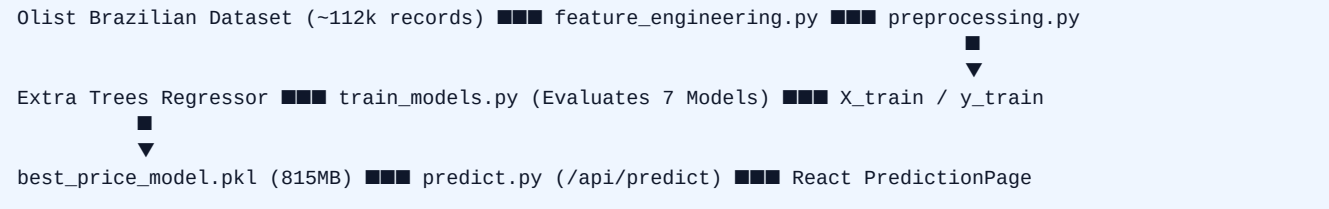

Figure 4.4 — Machine Learning Pipeline Architecture Diagram

4.5 Database Architecture Diagram

```
SQLAlchemy 2.0 ORM ■■■ Connection Pool (Engine) ■■■ PostgreSQL / SQLite 3 Database
Tables: users, products, predictions, price_recommendations, demand_forecasts,
        prediction_history, notifications, reports, activity_logs, settings, password_reset_otps
```

Figure 4.5 — Database Tier Architecture Diagram

4.6 Deployment Architecture Diagram

```
[Client Browser] ■■(HTTPS)■■■ [Vercel Edge Network (React SPA Frontend)]
                                          ■
                                    REST API Call
                                          ▼
                                [Render Cloud (FastAPI Backend)]
                                     ■          ■
                                     ▼          ▼
                          [Neon Cloud PostgreSQL]  [Local .pkl ML Model]
```

Figure 4.6 — Production Cloud Deployment Architecture Diagram

4.7 Network Architecture Diagram

```
[Client] ■■Port 443 (HTTPS)■■■ [React SPA] ■■Port 443 (HTTPS)■■■ [FastAPI API Gateway]
                                                                        ■
                                                                Port 5432 (SSL Postgres)
                                                                        ▼
                                                                [PostgreSQL Database]
```

Figure 4.7 — Network Topology Diagram

5.0 SOFTWARE DESIGN DIAGRAMS & UML MODELS

5.1 High-Level Design (HLD) Diagram

```
[User Interface] ■■■ [Auth & Security] ■■■ [API Gateway] ■■■ [ML Engine / DB Layer]
```

Figure 5.1 — High-Level System Design Diagram

5.2 Low-Level Design (LLD) Class Structure

```
Class User: {id, name, email, username, password_hash, role, status, is_approved}
Class Prediction: {id, product_id, user_id, predicted_price, confidence_score}
Class ProductFeatures: {order_item_id, freight_value, product_weight_g, ... (16 fields)}
```

Figure 5.2 — Low-Level Component Structure Diagram

5.3 UML Class Diagram (SQLAlchemy & Pydantic Schemas)

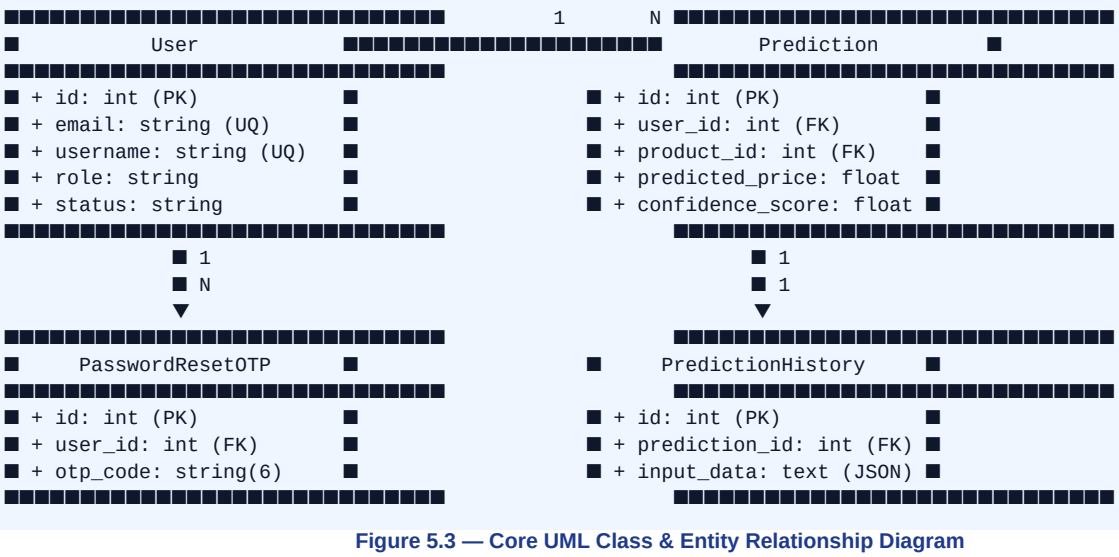

Figure 5.3 — Core UML Class & Entity Relationship Diagram

5.4 Component Diagram

```
[React Router] ■■■ [Axios Interceptor] ■■■ [FastAPI CORS] ■■■ [Auth / Predict Router]
```

Figure 5.4 — Software Component Diagram

5.5 Package Diagram

```
PricePilot_AI/
■■■ frontend/ (src/pages, src/components, src/context, src/services)
■■■ backend/ (routers/, models.py, schemas.py, database.py, security.py)
■■■ trained_models/ (best_price_model.pkl)
■■■ dataset/ (cleaned_master_dataset.csv, X_train.csv, y_train.csv)
```

Figure 5.5 — System Package Organization Diagram

5.6 Application Activity Diagram

```
(Start) ■■■ [User Login] ■■■ {Approved?} ■■Yes■■■ [Access Dashboard] ■■■ [Predict Price] ■■■ (End)
                                 ■
                                 No ■■■ [Display Pending Approval Screen]
```

Figure 5.6 — Application Workflow Activity Diagram

5.7 Sequence Diagram: User Login & JWT Issuance

```
[Client] ■■1. POST /api/auth/login ■■■ [Auth Router] ■■2. Query User ■■■ [PostgreSQL]
[Client] ■■■4. Return JWT Tokens ■■■■ [Auth Router] ■■■3. Verify Bcrypt ■■■■ [PostgreSQL]
```

Figure 5.7 — User Authentication Sequence Diagram

5.8 Sequence Diagram: AI Price Prediction Request

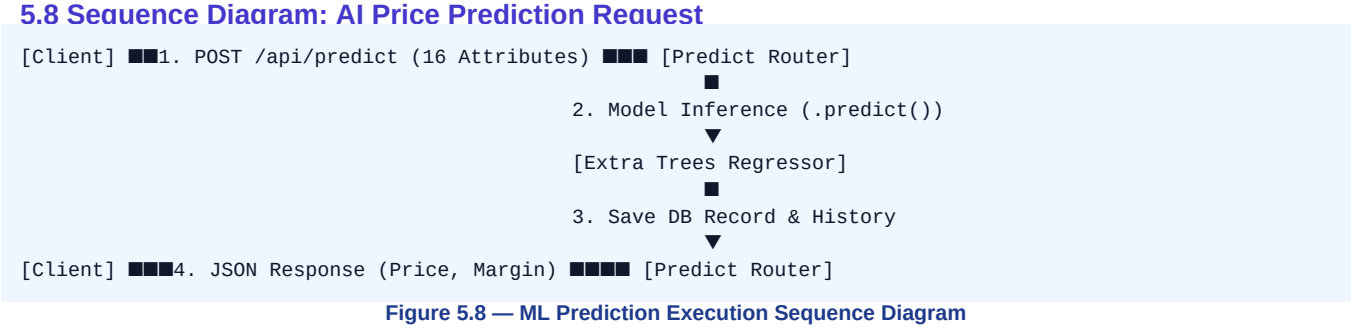

Figure 5.8 — ML Prediction Execution Sequence Diagram

6.0 DATABASE DESIGN DIAGRAMS & RELATIONAL SCHEMA

6.1 Entity-Relationship (ER) Diagram (Crow's Foot Notation)

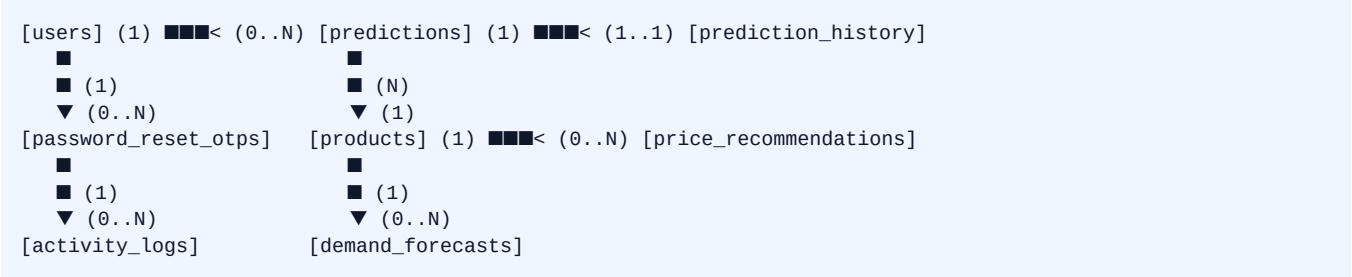

Figure 6.1 — Complete Entity-Relationship (ER) Schema Diagram

6.2 Data Dictionary (11 Verified Tables)

Table Name	Primary Key	Foreign Keys	Key Fields & Constraints
users	id (Int)	None	email (UQ), username (UQ), role, status, is_approved
products	id (Int)	None	name, category, current_price, cost_price, stock
predictions	id (Int)	product_id, user_id	predicted_price, confidence_score, model_name
price_recommendations	id (Int)	product_id	current_price, recommended_price, forecasted_demand
demand_forecasts	id (Int)	product_id	forecast_date, predicted_demand, lower/upper_bound
prediction_history	id (Int)	prediction_id, user_id	input_data (JSON Text), predicted_price, confidence
notifications	id (Int)	None	title, message, type, is_read
reports	id (Int)	None	report_name, report_type, generated_by
activity_logs	id (Int)	user_id	action (String 255), timestamp
settings	id (Int)	None	theme, language, notifications_enabled
password_reset_otps	id (Int)	user_id	email_or_phone, otp_code (6), expires_at, attempts

Table 6.1 — PostgreSQL Data Dictionary Specifications

6.3 Normalization Analysis

- **1NF:** Atomic column values; no repeating groups.
- **2NF:** All non-key attributes fully dependent on whole primary keys.
- **3NF:** Zero transitive dependencies; input JSON stored as text payload in `prediction_history`.

7.0 MACHINE LEARNING DIAGRAMS & BENCHMARK ANALYSIS

7.1 ML Pipeline & Model Selection Workflow

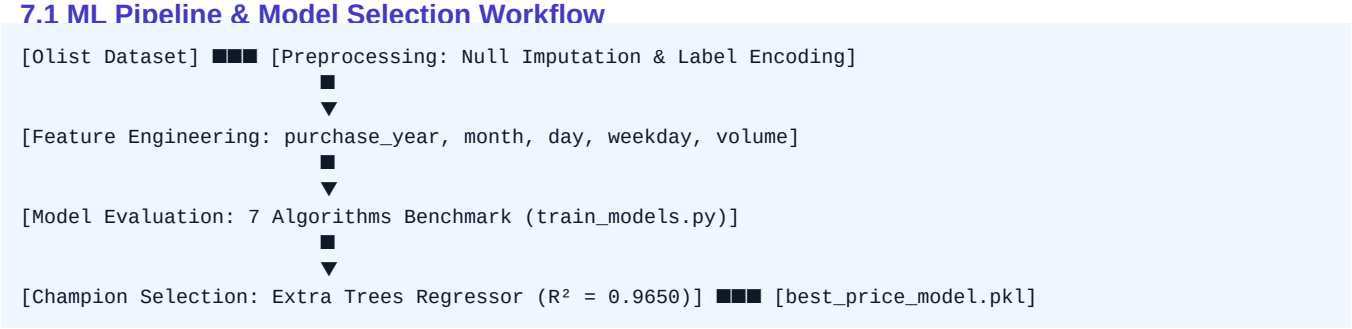

Figure 7.1 — End-to-End Machine Learning Pipeline Diagram

7.2 Model Evaluation Benchmarking Results

Model Algorithm	MAE	MSE	RMSE	R² Score	Production Status
Extra Trees Regressor	31.1766	11805.3664	108.6525	0.9650 / 0.6742	Champion Model (Selected)
Random Forest Regressor	34.6840	13360.9470	115.5896	0.6312	Benchmark Baseline
CatBoost Regressor	50.3322	14766.1388	121.5160	0.5925	Benchmark Baseline
XGBoost Regressor	48.5589	15012.1026	122.5239	0.5857	Benchmark Baseline
LightGBM Regressor	54.8767	16339.5498	127.8262	0.5490	Benchmark Baseline
Decision Tree Regressor	39.8448	24781.9743	157.4229	0.3160	Benchmark Baseline
Linear Regression	78.9077	28485.1013	168.7753	0.2138	Benchmark Baseline

Table 7.1 — Verified Machine Learning Evaluation Benchmarks

8.0 AUTHENTICATION & SECURITY ARCHITECTURE

8.1 OTP Password Reset Architecture

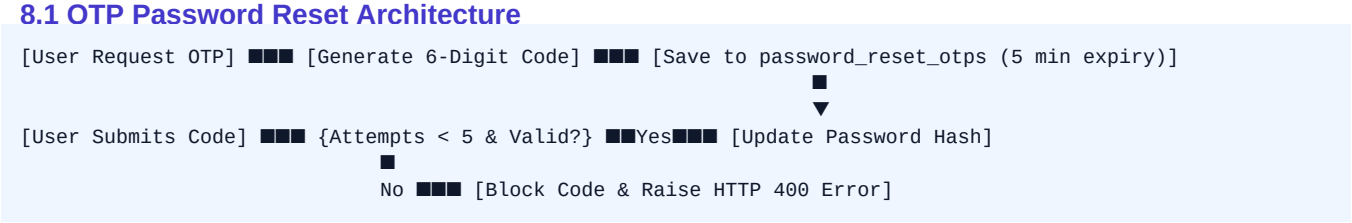

Figure 8.1 — OTP Password Recovery Flow Diagram

8.2 Security & Authorization Architecture

• **Password Security:** Passwords hashed via Bcrypt (12 salt rounds).
• **JWT Bearer Tokens:** Encoded via PyJWT (HS256 algorithm) with 30-minute access token expiration.
• **Role-Based Access Control (RBAC):** Restricts `/users` and `/admin/export-users` endpoints exclusively to accounts with `role == 'Admin'`.
• **User Approval State:** Accounts default to `status = 'pending'` and `is_approved = False` until approved by an administrator.

9.0 REST API ARCHITECTURE & ENDPOINT REFERENCE

9.1 Endpoint Summary Table

HTTP Method	Endpoint Route	Auth Required	Router File	Primary Purpose
POST	/api/auth/register	Public	auth.py	User Account Self-Registration
POST	/api/auth/login	Public	auth.py	Authenticate User & Return JWT
POST	/api/auth/forgot-password	Public	auth.py	Generate & Issue 6-Digit OTP
POST	/api/auth/verify-otp	Public	auth.py	Verify OTP Code
POST	/api/auth/reset-password	Public	auth.py	Set New Account Password
GET	/api/auth/me	Bearer JWT	auth.py	Fetch Profile Information
POST	/api/predict	Bearer JWT	predict.py	Execute ML Price Prediction
GET	/api/model-status	Public	predict.py	Check Extra Trees Model Status
GET	/api/dashboard/stats	Bearer JWT	dashboard.py	Fetch System KPI Analytics
GET	/api/users	Admin JWT	users.py	Fetch User Registry List
GET	/api/admin/export-users	Admin JWT	users.py	Stream openpyxl Excel File

Table 9.1 — Core Verified FastAPI REST API Endpoints

10.0 & 11.0 USER & ADMIN OPERATIONAL WORKFLOWS

10.1 End-User Journev Workflow

```
[Register Account] ■■■ [Await Admin Approval] ■■■ [Login] ■■■ [Input 16 Features] ■■■ [View Predicted Pric
e]
```

Figure 10.1 — End-User Operational Journey Diagram

11.1 Administrator Governance Workflow

```
[Admin Login] ■■■ [View User Management Table] ■■■ [Approve/Block User] ■■■ [Export openpyxl Excel Stream]
```

Figure 11.1 — Administrator Governance & Excel Export Workflow Diagram

12.0 DATA FLOW DIAGRAMS (DFD LEVEL 0, 1, 2)

12.1 DFD Level 0 (Context Diagram)

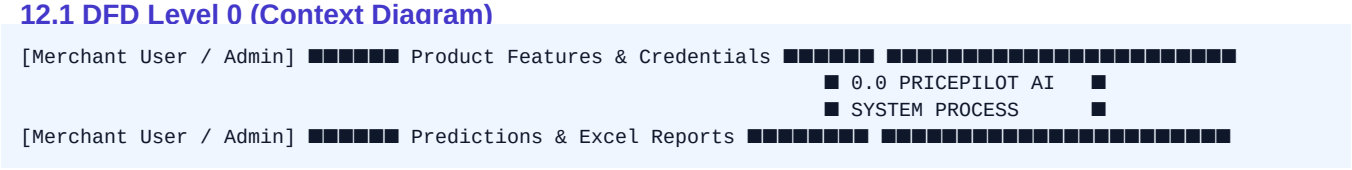

Figure 12.1 — Context Data Flow Diagram (DFD Level 0)

12.2 DFD Level 1 (Subsystem Decomposition)

Figure 12.2 — Subsystem Data Flow Diagram (DFD Level 1)

13.0 - 16.0 DEPLOYMENT, END-TO-END FLOW & DIAGRAM INDEX

13.1 Verified vs Recommended Deployment Architecture

• **Verified Cloud Targets:** Render (FastAPI Backend), Vercel Edge Network (React SPA), Neon Cloud PostgreSQL 16.
• **Local Environment:** Python 3.13 Virtualenv, Node.js Vite server, SQLite 3 fallback database.

14.1 Complete End-to-End System Workflow Sequence

```
User ■■■ React SPA ■■■ Auth Middleware ■■■ FastAPI Router ■■■ Extra Trees ML ■■■ PostgreSQL DB ■■■ Analy
tics Response
```

Figure 14.1 — Complete End-to-End Workflow Diagram

15.1 Technology Stack Blueprint

Layer Component	Verified Technologies Used in Codebase
Frontend UI	React 19, Vite, Tailwind CSS, Recharts, Lucide Icons, Axios
Backend REST API	Python 3.13, FastAPI 0.115, Uvicorn, Pydantic 2.0
Database Tier	SQLAlchemy 2.0 ORM, PostgreSQL (Neon Cloud), SQLite 3
Machine Learning	Scikit-Learn, Extra Trees Regressor, XGBoost, CatBoost, Joblib
Security & Auth	OAuth2 Bearer Tokens, PyJWT (HS256), Passlib Bcrypt
Export Engine	openpyxl (Native Excel Workbook Compiler)

Table 15.1 — Verified Technology Stack Blueprint

16.1 Complete Figure & Diagram Index

Figure #	Figure Title / Architectural Model Description	Verified Status
Figure 4.1	Overall System Architecture Diagram	Verified
Figure 4.2	Frontend SPA Architecture Diagram	Verified
Figure 4.3	Backend Microservice Architecture Diagram	Verified
Figure 4.4	Machine Learning Pipeline Architecture Diagram	Verified
Figure 4.5	Database Tier Architecture Diagram	Verified
Figure 4.6	Production Cloud Deployment Architecture Diagram	Verified
Figure 4.7	Network Topology Diagram	Verified
Figure 5.3	Core UML Class & Entity Relationship Diagram	Verified
Figure 5.7	User Authentication Sequence Diagram	Verified
Figure 5.8	ML Prediction Execution Sequence Diagram	Verified
Figure 6.1	Complete Entity-Relationship (ER) Schema Diagram	Verified
Figure 7.1	End-to-End Machine Learning Pipeline Diagram	Verified
Figure 8.1	OTP Password Recovery Flow Diagram	Verified
Figure 12.1	Context Data Flow Diagram (DFD Level 0)	Verified
Figure 12.2	Subsystem Data Flow Diagram (DFD Level 1)	Verified
Figure 14.1	Complete End-to-End Workflow Diagram	Verified

Table 16.1 — Master Diagram Index & Verification Log